

SQL DBA CHAMPS

Learning Path Series

Amazon Aurora

From Basics to Advanced — Step by Step

Architecture · Scaling · HA/DR · Performance · Security · Real-Time Scenarios

A hands-on teaching guide for SQL Server DBAs moving into AWS data platforms

Prepared by Praveen Madupu

Table of Contents

Table of Contents	2
Part 1 — Foundations: What Aurora Is and Why It Exists	3
1.1 What is Amazon Aurora?	3
1.2 Aurora vs RDS vs Self-Managed	3
1.3 The Aurora Storage Architecture (the heart of it)	3
Part 2 — Core Building Blocks	5
2.1 Cluster and DB instances	5
2.2 The four endpoint types (know these cold)	5
2.3 Create a cluster (CLI)	5
Part 3 — Compute Options and Scaling	6
3.1 Provisioned vs Serverless v2	6
3.2 Aurora Auto Scaling for read replicas	6
Part 4 — High Availability & Disaster Recovery	7
4.1 Automatic failover & promotion tiers	7
4.2 Aurora Global Database (cross-region DR + low latency)	7
4.3 Backtrack (Aurora MySQL) — rewind time without a restore	7
4.4 Fast cloning (copy-on-write)	7
4.5 Backups & Point-in-Time Recovery (PITR)	8
Part 5 — Performance & Monitoring	9
5.1 Scaling reads	9
5.2 Parallel Query (Aurora MySQL)	9
5.3 Monitoring toolkit	9
Part 6 — Security Essentials	10
Part 7 — Advanced Features	11
7.1 Babelfish for Aurora PostgreSQL (SQL Server compatibility)	11
7.2 Aurora I/O-Optimized vs Standard	11
7.3 Blue/Green deployments	11
7.4 Zero-ETL integration with Amazon Redshift	11
7.5 Aurora Machine Learning	11
Part 8 — Getting Data Into Aurora	12
Part 9 — Real-Time Scenarios	13
Part 10 — Quick Reference	14
10.1 Feature cheat-sheet	14
10.2 Numbers worth memorising	14

Part 1 — Foundations: What Aurora Is and Why It Exists

LEARNING GOAL

By the end of this part you can explain what Aurora is, how it differs from RDS and self-managed databases, and the storage architecture that makes it special.

1.1 What is Amazon Aurora?

Amazon Aurora is a **fully managed, cloud-native relational database** that is wire-compatible with **MySQL and PostgreSQL**. You keep using the same drivers, SQL and tools, but AWS re-engineered the storage and replication layer for the cloud — giving much higher throughput, built-in fault tolerance and automatic storage scaling.

- **Aurora MySQL** — drop-in compatible with MySQL (community editions).
- **Aurora PostgreSQL** — drop-in compatible with PostgreSQL, and the base for Babelfish (SQL Server compatibility).

CONCEPT — The one-sentence definition

Aurora = the MySQL/PostgreSQL engine you know, running on a distributed, self-healing, auto-scaling storage layer that AWS manages for you.

1.2 Aurora vs RDS vs Self-Managed

Dimension	Self-managed (EC2 / on-prem)	Amazon RDS	Amazon Aurora
Who manages OS/patching	You	AWS	AWS
Storage	You provision & grow manually	Provisioned, some autoscaling	Auto-grows 10 GB at a time, up to 128 TiB
Replication	You build	Physical/logical replicas	6-way distributed, log-based
Read replicas	Manual	Up to 5–15	Up to 15 Aurora Replicas, shared storage
Failover	You script	~60–120s	Typically under 30s
Best for	Full control	Standard managed DBs	High-throughput, HA-critical workloads

1.3 The Aurora Storage Architecture (the heart of it)

Aurora's defining idea is separating compute from storage. Your DB instances (compute) are query engines; the data lives in a shared, distributed cluster volume.

CONCEPT — 6 copies across 3 Availability Zones

Aurora keeps 6 copies of your data — 2 in each of 3 AZs.

Writes use a 4-of-6 quorum (a write is durable once 4 copies acknowledge).

Reads use a 3-of-6 quorum. This survives losing an entire AZ plus one more copy without data loss.

Because storage is a separate distributed service, it is self-healing (bad segments are repaired from peers) and grows automatically in 10 GB chunks as data increases — no manual resizing.

Writer + up to 15 Aurora Replicas (compute)

```
      |   |   |
+-----+-----+-----+
|   Shared Distributed Cluster Volume   |
| AZ-a [copy1 copy2]  AZ-b [copy3 copy4] |
|           AZ-c [copy5 copy6]           |
+-----+
Writes: 4/6 quorum   Reads: 3/6 quorum   Auto-heal + auto-grow
```

KEY TAKEAWAY

Aurora sends redo log records to storage, not full data pages. The storage layer materialises pages. This is why Aurora writes far less over the network and scales better than traditional replication.

Part 2 — Core Building Blocks

LEARNING GOAL

Learn the objects you actually create and connect to: the cluster, DB instances, and the four endpoint types.

2.1 Cluster and DB instances

- **DB cluster** — the logical unit: one shared cluster volume plus one or more instances.
- **Writer (primary) instance** — the single instance that accepts writes.
- **Aurora Replicas (readers)** — up to 15 read-only instances sharing the same storage; used for read scaling and as failover targets.

2.2 The four endpoint types (know these cold)

Endpoint	Points to	Use it for
Cluster (writer) endpoint	Always the current writer	All write traffic; follows failover automatically
Reader endpoint	Load-balances across all readers	Read-only queries; scales with replicas
Custom endpoint	A chosen subset of instances	Routing (e.g. analytics to bigger instances)
Instance endpoint	One specific instance	Diagnostics / targeted connections

WATCH OUT — Common mistake

Applications should point writes at the CLUSTER endpoint and reads at the READER endpoint — never hard-code an instance endpoint. Hard-coding breaks automatic failover routing.

2.3 Create a cluster (CLI)

```
# 1) Create the cluster (Aurora PostgreSQL-compatible)
aws rds create-db-cluster \
  --db-cluster-identifier champs-aurora \
  --engine aurora-postgresql \
  --master-username champsadmin \
  --manage-master-user-password \
  --vpc-security-group-ids sg-0abc123 \
  --db-subnet-group-name champs-subnet-grp

# 2) Add the writer instance
aws rds create-db-instance \
  --db-instance-identifier champs-aurora-writer \
  --db-cluster-identifier champs-aurora \
  --engine aurora-postgresql \
  --db-instance-class db.r6g.large

# 3) Add a read replica (reader)
aws rds create-db-instance \
  --db-instance-identifier champs-aurora-reader-1 \
  --db-cluster-identifier champs-aurora \
  --engine aurora-postgresql \
  --db-instance-class db.r6g.large
```

Part 3 — Compute Options and Scaling

LEARNING GOAL

Choose between provisioned and Serverless v2, and scale reads automatically.

3.1 Provisioned vs Serverless v2

	Provisioned	Aurora Serverless v2
You pick	A fixed instance class (e.g. db.r6g.xlarge)	A capacity range in ACUs (min–max)
Scaling	Manual / vertical resize	Automatic, fine-grained, in-place
Unit	Instance size	ACU (~2 GiB memory each), 0.5 increments
Best for	Steady, predictable load	Spiky, variable, or unpredictable load
Cost model	Pay for the instance running	Pay for ACUs consumed per second

CONCEPT — What is an ACU?

An Aurora Capacity Unit is a bundle of compute + memory (roughly 2 GiB RAM plus associated CPU/network). Serverless v2 adjusts ACUs up and down automatically between your configured minimum and maximum, in 0.5 steps, without dropping connections.

You can even mix them: a provisioned writer with Serverless v2 readers, or vice-versa, in the same cluster.

3.2 Aurora Auto Scaling for read replicas

Aurora Auto Scaling adds or removes Aurora Replicas automatically based on a target metric (e.g. average CPU or connections). Great for read-heavy traffic that peaks and troughs.

```
# Register the cluster's replica count as a scalable target
aws application-autoscaling register-scalable-target \
  --service-namespace rds \
  --resource-id cluster:champs-aurora \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --min-capacity 1 --max-capacity 8

# Target-tracking policy: keep average reader CPU at 60%
aws application-autoscaling put-scaling-policy \
  --policy-name champs-reader-cpu \
  --service-namespace rds \
  --resource-id cluster:champs-aurora \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration \
    '{"TargetValue":60.0,"PredefinedMetricSpecification":\
      {"PredefinedMetricType":"RDSReaderAverageCPUUtilization"}}'
```

Part 4 — High Availability & Disaster Recovery

LEARNING GOAL

Understand automatic failover, Global Database, Backtrack, cloning, and backups/PITR — the features interviews and incidents both hinge on.

4.1 Automatic failover & promotion tiers

If the writer fails, Aurora promotes a reader to writer — usually in under 30 seconds. Which reader wins is decided by the promotion (failover) priority tier.

- Tiers run 0 (highest priority) to 15 (lowest).
- Aurora promotes the reader in the lowest-numbered tier; ties are broken by the largest instance size.
- Set critical, well-sized replicas to tier 0 or 1 so they're chosen first.

KEY TAKEAWAY

Because replicas share the same storage as the writer, failover doesn't need to copy data — it just repoints. That's why it's fast and loses little to no data.

4.2 Aurora Global Database (cross-region DR + low latency)

A Global Database spans one primary region and one or more secondary regions. Aurora replicates via the storage layer with typical cross-region lag under a second.

- Read locally in each region for low latency to global users.
- Fast cross-region disaster recovery: promote a secondary region, typically with an RTO of about a minute and very low RPO.
- Offloads replication from the database engine to the storage layer, so it barely taxes the primary.

CONCEPT — Global Database vs a plain cross-region replica

Global Database uses dedicated storage-level replication (sub-second lag, minimal primary overhead) and is purpose-built for DR and global reads. A traditional read replica uses engine-level replication and lags more under load.

4.3 Backtrack (Aurora MySQL) — rewind time without a restore

Backtrack lets you rewind an Aurora MySQL cluster to a point in time in seconds — without restoring from a backup — by rolling the storage back. Ideal for undoing a bad deployment or accidental mass update.

WATCH OUT — Scope

Backtrack is Aurora MySQL only (not PostgreSQL) and must be enabled at cluster creation with a target backtrack window. It rewinds the whole cluster, not a single table.

4.4 Fast cloning (copy-on-write)

Aurora can clone a cluster almost instantly using copy-on-write: the clone initially shares the source's storage pages and only diverges as either side changes data. You get a full, writable copy for testing without duplicating terabytes.

- Near-instant creation regardless of database size.
- Only pay extra storage for pages that change after cloning.
- Perfect for dev/test, schema-change rehearsals, and reporting sandboxes.

4.5 Backups & Point-in-Time Recovery (PITR)

Aurora continuously backs up the cluster volume to Amazon S3 and lets you restore to any second within the retention window (up to 35 days). Backups are automatic and don't impact performance the way a traditional backup job does.

```
# Restore the cluster to a point in time (creates a new cluster)
aws rds restore-db-cluster-to-point-in-time \
  --db-cluster-identifier champs-aurora-pitr \
  --source-db-cluster-identifier champs-aurora \
  --restore-to-time 2026-07-12T14:30:00Z
```

Part 5 — Performance & Monitoring

LEARNING GOAL

Scale and observe Aurora: reader load-balancing, parallel query, and the monitoring tools you'll actually use.

5.1 Scaling reads

- **Reader endpoint** load-balances connections across replicas — point read-only workloads here.
- **Auto Scaling** adds replicas as read load rises (Part 3).
- **Custom endpoints** route heavy analytical reads to larger instances, isolating them from OLTP.

5.2 Parallel Query (Aurora MySQL)

Parallel Query pushes parts of a query (filtering, aggregation) down into the distributed storage layer, running them in parallel across storage nodes. Large analytical scans on big tables run dramatically faster without a separate analytics system.

5.3 Monitoring toolkit

Tool	What it shows	When to reach for it
Amazon CloudWatch	CPU, connections, IOPS, replica lag, ACUs	Baseline metrics & alarms
Performance Insights	Top SQL, wait events, DB load over time	Find what's actually slow / blocking
Enhanced Monitoring	OS-level metrics at 1–60s granularity	Deep host-level troubleshooting
Database Activity Streams	Near-real-time audit of activity	Security/compliance auditing

KEY TAKEAWAY

Performance Insights is your first stop for "the database is slow" — it ranks queries by DB load and shows the wait events behind them, much like Query Store did for you on SQL Server.

Part 6 — Security Essentials

LEARNING GOAL

Lock Aurora down: network isolation, encryption, and identity.

- **Network** — Aurora runs inside a VPC; security groups control access, and it should sit in private subnets with no public IP.
- **Encryption at rest** — AWS KMS encrypts the cluster volume, automated backups, snapshots and replicas. Enable it at creation.
- **Encryption in transit** — TLS/SSL for client connections using the AWS-provided certificate bundle.
- **IAM database authentication** — authenticate with short-lived IAM tokens instead of passwords for supported connections.
- **Secrets Manager** — store and automatically rotate the master credentials (the `--manage-master-user-password` flag wires this up).
- **Auditing** — Database Activity Streams and engine audit logs feed your SIEM.

WATCH OUT

You cannot turn on at-rest encryption for an existing unencrypted cluster in place — you must restore/copy a snapshot into a new encrypted cluster. Decide encryption up front.

Part 7 — Advanced Features

LEARNING GOAL

The differentiators worth knowing: Babelfish, I/O-Optimized, Blue/Green, Zero-ETL, and Aurora ML.

7.1 Babelfish for Aurora PostgreSQL (SQL Server compatibility)

Babelfish lets Aurora PostgreSQL understand the SQL Server wire protocol (TDS, port 1433) and T-SQL. A SQL Server application can connect to Aurora with **little or no code change** — the key to migrating off SQL Server licensing while protecting your app investment.

KEY TAKEAWAY — Why this matters to you

For a SQL Server DBA, Babelfish is the bridge: your T-SQL, TDS drivers and much of your app keep working, but the engine underneath is licence-free Aurora PostgreSQL. Pair with SCT + DMS to convert and move data.

7.2 Aurora I/O-Optimized vs Standard

Aurora has two storage configurations. Standard charges separately for I/O; I/O-Optimized removes per-request I/O charges for higher, predictable compute/storage pricing.

	Aurora Standard	Aurora I/O-Optimized
I/O charges	Pay per I/O request	No per-I/O charges
Best for	Low-to-moderate I/O workloads	I/O-heavy workloads (I/O a large share of bill)
Pricing	Lower base, variable I/O	Higher base, predictable

CONCEPT — Rule of thumb

If I/O is a large share of your Aurora bill (often cited around 25%+), I/O-Optimized is usually cheaper and far more predictable. You can switch configuration on an existing cluster.

7.3 Blue/Green deployments

A Blue/Green deployment creates a synchronised green (staging) copy of your cluster. You apply changes — a major version upgrade, schema change, parameter change — to green, test it, then switch over in about a minute with safeguards that block switchover if replication is unhealthy.

7.4 Zero-ETL integration with Amazon Redshift

Zero-ETL continuously replicates Aurora data into Amazon Redshift for analytics with no ETL pipeline to build or maintain — transactional data becomes available for near-real-time analytics automatically.

7.5 Aurora Machine Learning

Aurora ML lets you call Amazon SageMaker and Amazon Comprehend directly from SQL, so applications can enrich queries with predictions (fraud scoring, sentiment, recommendations) without moving data out.

Part 8 — Getting Data Into Aurora

- **From RDS MySQL/PostgreSQL** — create an Aurora read replica of the RDS instance, let it catch up, then promote it to a standalone Aurora cluster.
- **From on-prem / other engines** — use AWS SCT to convert schema (for heterogeneous, e.g. SQL Server → Aurora PostgreSQL) and AWS DMS to move data with CDC for minimal downtime.
- **Snapshots** — restore an Aurora snapshot (or migrate a compatible RDS snapshot) into a new cluster.
- **Bulk load** — load from Amazon S3 (e.g. the `aws_s3` extension on Aurora PostgreSQL, or `LOAD DATA FROM S3` on Aurora MySQL) for large initial imports.

KEY TAKEAWAY

Migrating a SQL Server workload? The path is SCT (convert schema + assess) → DMS full-load + CDC (move data, minimal downtime) → optionally Babelfish (keep T-SQL apps working). This ties directly to the AWS migration flow you already know.

Part 9 — Real-Time Scenarios

Applied situations that connect the features above to decisions you'd make on the job.

SCENARIO Read-heavy app is overloading the primary

Problem: An e-commerce catalog gets 90% reads; the writer's CPU is pegged at peak.

Solution: Move read traffic to the reader endpoint, add Aurora Replicas, and enable Aurora Auto Scaling to add/remove readers on a CPU target. Route heavy report queries to a custom endpoint on larger instances so they don't starve OLTP reads.

Why it works: Replicas share storage (no lag from copying data) and the reader endpoint load-balances automatically.

SCENARIO Global users complain about latency + you need cross-region DR

Problem: Users in Europe and Asia hit a US-only database; leadership also wants regional DR.

Solution: Deploy an Aurora Global Database with secondary regions near your users. They read locally (sub-second replication), and if the primary region fails you promote a secondary with roughly a one-minute RTO.

Key point: Storage-level replication keeps overhead off the primary engine — unlike a traditional read replica.

SCENARIO Unpredictable, spiky workload (new product launch)

Problem: Traffic could be 10x or flat — you can't size a fixed instance well and don't want to overpay.

Solution: Use Aurora Serverless v2 with a sensible min–max ACU range. Capacity scales in 0.5-ACU steps in-place as load changes, and you pay per second of ACUs used.

Watch: Set the maximum high enough to absorb the spike, and the minimum high enough to avoid sluggish scale-ups on the first burst.

SCENARIO A bad deployment corrupted data 20 minutes ago (Aurora MySQL)

Problem: A faulty release ran an UPDATE without a WHERE clause across a key table.

Fast fix: If Backtrack is enabled, rewind the cluster a few minutes to just before the deploy — seconds, no restore needed.

Alternative: No Backtrack (e.g. Aurora PostgreSQL)? Use PITR to restore to a new cluster just before the event, or fast-clone and recover the affected table.

SCENARIO The Aurora bill jumped and I/O is the culprit

Problem: An analytics-heavy month drove I/O charges way up on Aurora Standard.

Solution: Switch the cluster to Aurora I/O-Optimized, which removes per-I/O charges for a higher but predictable base price. When I/O is a large share of the bill, this is typically cheaper and easier to forecast.

SCENARIO Migrate a licensed SQL Server app to cut cost

Problem: Recurring SQL Server licence spend; app is heavy on T-SQL and TDS drivers.

Solution: Refactor to Aurora PostgreSQL with Babelfish. Convert schema/code with SCT, move data with DMS (full-load + CDC), and point the app at the Babelfish TDS endpoint so most T-SQL and drivers keep working.

Payoff: Licence-free engine plus managed HA/backups, with far less application rewrite than a raw PostgreSQL port.

SCENARIO Safely test a major version upgrade in production

Problem: You must upgrade the engine major version but can't risk the live cluster.

Solution: Create a Blue/Green deployment, apply the upgrade on green, validate with real workloads/tests, then switch over in about a minute. Switchover is blocked if replication to green is unhealthy, protecting you from a half-baked cutover.

Part 10 — Quick Reference

10.1 Feature cheat-sheet

Need	Aurora feature
Scale reads	Aurora Replicas + reader endpoint + Auto Scaling
Handle spiky load	Serverless v2 (ACU range)
Cross-region DR / global reads	Aurora Global Database
Instantly undo bad change (MySQL)	Backtrack
Recover to a point in time	Continuous backup + PITR
Instant test copy	Fast clone (copy-on-write)
Run SQL Server apps	Babelfish (Aurora PostgreSQL)
Predictable cost on heavy I/O	Aurora I/O-Optimized
Safe upgrades / changes	Blue/Green deployments
Analytics without ETL	Zero-ETL to Redshift
Find slow queries	Performance Insights

10.2 Numbers worth memorising

- 6 copies of data, across 3 AZs (2 per AZ).
- Write quorum 4/6; read quorum 3/6.
- Storage grows in 10 GB increments, up to 128 TiB.
- Up to 15 Aurora Replicas per cluster.
- Failover typically under 30 seconds.
- Backup retention (PITR) up to 35 days.
- Serverless v2: 1 ACU $\approx$ 2 GiB memory; scales in 0.5-ACU steps.

WATCH OUT — Verify current limits

AWS raises limits and adds features regularly (max storage, secondary-region counts, engine support). For anything you'll state as a hard number in an interview or design doc, confirm it against the current AWS Aurora documentation.